

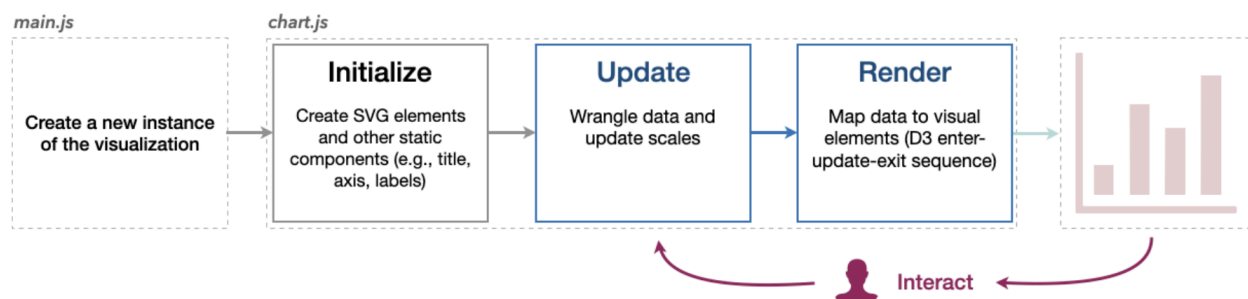
In this tutorial, we are going to look at a way to structure a complex visualization project. You don't have to structure a visualization project this way, but it is a good idea- your code will be more re-usable, your code will be easier to debug, and your code will be easier to extend for future applications.

Reusable components

How to avoid having one giant vis file?

Here is one way to structure a vis project, to avoid having a giant block of code. This becomes especially important in projects with multiple visualizations and with interactions- eg. the projects we will do in this class.

You can structure your code in a way where you have classes with each chart type, with common functions for initializing, updating and rendering.



With more recent versions of Javascript, you can create object-oriented code.

First, you create a class for your chart, and initialize it with size parameters and the parent element you are embedding it in.

```
// ES6 Class
class BarChart {

  constructor(_config) {
    this.config = {
      parentElement: _config.parentElement,
      containerWidth: _config.containerWidth || 500,
      containerHeight: _config.containerHeight || 140,
      margin: { top: 10, bottom: 30, right: 10, left: 30 }
    }
  }

  this.initVis();
}

initVis() {
  //setting up svg elements and other components- titles, etc
}
```

```

    // - things that do not change when the user interacts
    this.updateVis();
  }

  updateVis() {
    // things that may update when users interact
    // - may include data processing/wrangling
    // - may include setting the input domain on your scales
    this.renderVis();
  }

  renderVis() {
    // map data to visual elements
    // - d3 enter-update-exit sequence
  }
}

```

To create the bar chart, you use the keyword 'new' , and a constructor which passes the needed parameters to initialize the class in a 'config' object.

```

// Create an instance (for example in main.js)
let barchart = new BarChart({
  'parentElement': '#bar-chart-container',
  'containerHeight': 400
});

```

Initialization can involve setting up your svg, within the parent container that you passed as a parameter into your new chart.

```

initVis() {
  let vis = this;

  vis.svg = d3.select(vis.config.parentElement).append('svg');
}

```

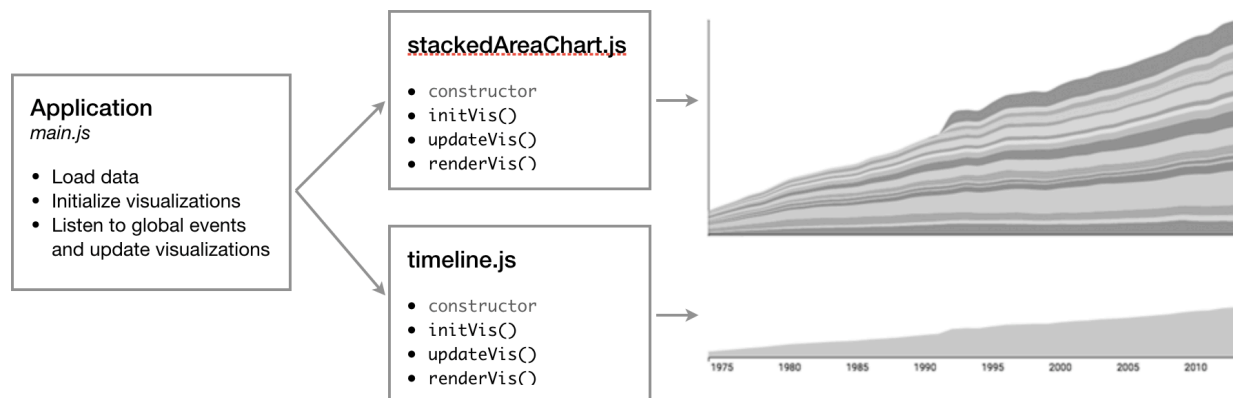
```

    vis.chart = vis.svg.append('g')
      .attr('transform',
`translate(${vis.config.margin.left},${vis.config.margin.top})`);
    // ...
  }

```

For now, since our charts have been static, you could put everything in `init`. Later, as we add interactivity, you can think about what needs to go in `'init'` vs `'update'` vs `'render'`, depending on the kinds of interactions you create.

You can in complex projects create object instances for the chart type classes in the file `main.js`, which can be the entry point for your application. You can load the data in `main.js`, and then pass it to the chart objects.



Reusable component tip:

You may want to create a variable name, to use throughout the code, that is assigned to an inline function that returns the parameter in the data objects for positional variables or color variables. You will use these a lot.

```

vis.xValue = d => d.date;

vis.yValue = d => d.close;

```

If you assign them in your initialization code, you can reusable them easily. You could also pass these as parameters in your constructor.

You can use the code like this:

```
// Set the scale input domains  
vis.xScale.domain(d3.extent(vis.data, vis.xValue));  
vis.yScale.domain(d3.extent(vis.data, vis.yValue));
```

Credits:

Adapted from

https://github.com/UBC-InfoVis/2021-436V-tutorials/tree/master/2_D3_Tutorial#making-line-and-area-charts